

# Typing Rules, Domain Semantics, and Pi Injectivity for Dependent Type Theory with $\mathbf{U} : \mathbf{U}$

## 1 Judgments

We work with three mutually defined judgment forms:

|                           |                                                  |
|---------------------------|--------------------------------------------------|
| $\Gamma \text{ ctx}$      | $\Gamma$ is a well-formed context                |
| $\Gamma \vdash M : A$     | $M$ has type $A$ in context $\Gamma$             |
| $\Gamma \vdash M = N : A$ | $M$ and $N$ are definitionally equal at type $A$ |

Contexts are snoc-lists:  $\Gamma ::= \cdot \mid \Gamma, x:A$ .

## 2 Well-formed contexts

$$\frac{}{\cdot \text{ ctx}} \qquad \frac{\Gamma \vdash A : \mathbf{U}}{\Gamma, x:A \text{ ctx}}$$

## 3 Typing rules

$$\begin{array}{c} \text{VAR} \\ \frac{\Gamma \text{ ctx} \quad (x:A) \in \Gamma}{\Gamma \vdash x : A} \end{array} \qquad \begin{array}{c} \text{CONV} \\ \frac{\Gamma \vdash M : A \quad \Gamma \vdash A = B : \mathbf{U} \quad \Gamma \vdash B : \mathbf{U}}{\Gamma \vdash M : B} \end{array} \qquad \begin{array}{c} \mathbf{U} \\ \frac{\Gamma \text{ ctx}}{\Gamma \vdash \mathbf{U} : \mathbf{U}} \end{array}$$
  

$$\begin{array}{c} \text{PI} \\ \frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U}}{\Gamma \vdash \Pi(x:A)B : \mathbf{U}} \end{array} \qquad \begin{array}{c} \text{LAM} \\ \frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda(x:A)M : \Pi(x:A)B} \end{array}$$
  

$$\begin{array}{c} \text{APP} \\ \frac{\Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B a} \end{array}$$

## 4 Conversion rules

$$\begin{array}{c}
\text{REFL} \\
\frac{\Gamma \vdash M : A}{\Gamma \vdash M = M : A} \\
\\
\text{SYM} \\
\frac{\Gamma \vdash M = N : A}{\Gamma \vdash N = M : A} \\
\\
\text{TRANS} \\
\frac{\Gamma \vdash M = N : A \quad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A} \\
\\
\text{CONV-EQ} \\
\frac{\Gamma \vdash M = N : A \quad \Gamma \vdash A = B : \mathbb{U} \quad \Gamma \vdash B : \mathbb{U}}{\Gamma \vdash M = N : B} \\
\\
\text{BETA} \\
\frac{\Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma, x:A \vdash M : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x:A)M) a = M a : B a} \\
\\
\text{PI-CONG} \\
\frac{\Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \mathbb{U}}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : \mathbb{U}}
\end{array}$$

Note that in PI-CONG the codomain premise compares  $B$  and  $B'$  in the context extended by the *left* domain  $A$ , not by  $A'$ . This is the standard convention: since  $A = A' : \mathbb{U}$  holds, one could equivalently use  $A'$  (via the conversion rule on contexts), but fixing the left side avoids a mutual dependency between the congruence rule and context conversion.

We write  $M^+$  for the weakening of a term  $M$  by one fresh variable, i.e.  $M^+$  is  $M$  viewed in the extended context  $\Gamma, x:A$  where it does not depend on  $x$ .

$$\begin{array}{c}
\text{FUNEXT} \\
\frac{\Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash f^+ x = g^+ x : B \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash g : \Pi(x:A)B}{\Gamma \vdash f = g : \Pi(x:A)B} \\
\\
\text{APP-FUN} \\
\frac{\Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash f = f' : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a = f' a : B a} \\
\\
\text{APP-ARG} \\
\frac{\Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a = a' : A}{\Gamma \vdash f a = f a' : B a}
\end{array}$$

**Theorem 1** (Eta-conversion). *If  $\Gamma \vdash A : \mathbb{U}$ ,  $\Gamma, x:A \vdash B : \mathbb{U}$ , and  $\Gamma \vdash f : \Pi(x:A)B$ , then*

$$\Gamma \vdash f = \lambda(x:A)f^+ x : \Pi(x:A)B.$$

*Proof.* By FUNEXT it suffices to show  $\Gamma, x:A \vdash f^+ x = (\lambda(x:A)f^+ x)^+ x : B$ . Since weakening distributes under  $\lambda$ , we have  $(\lambda(x:A)M)^+ = \lambda(x:A^+)M^+$  (where the inner weakening acts on the next-to-last variable, skipping the binder). The right-hand side is therefore a  $\beta$ -redex. Applying BETA and the substitution identity  $M^+ x_0 = M$  gives the result. Formalised in `EtaConversion.agda`.  $\square$

## 5 Head reduction

We define head reduction  $M \rightsquigarrow N$  as the compatible closure of  $\beta$ -reduction at the head:

$$\begin{array}{c}
\text{HR-BETA} \\
\frac{}{(\lambda(x:A)M) a \rightsquigarrow M a} \\
\\
\text{HR-APP} \\
\frac{M \rightsquigarrow M'}{M a \rightsquigarrow M' a}
\end{array}$$

We write  $M \rightsquigarrow^* N$  for the reflexive-transitive closure. The validity relation uses a wrapper  $\text{Red } \Gamma \ M \ N \ A$ , which carries only a proof of  $M \rightsquigarrow^* N$ ; the context  $\Gamma$  and type  $A$  are phantom parameters kept for inference convenience. In [1], the reduction relation is defined as a subrelation of conversion, requiring both  $M \rightsquigarrow^* N$  and  $\Gamma \vdash M = N : A$ . Here, reduction is purely syntactic: the conversion witnesses are instead stored directly at the leaves of the bundled validity relation ( $\text{Val2}/\text{EqVal2}$ ), so the  $\text{Red}$  wrapper need not carry them.

## 6 Finite elements and the information ordering

Following Coquand–Huber [1], we interpret types and terms as ideal-domain elements, approximated from below by *finite elements*. The set  $\text{FinEl}$  and its auxiliary  $\text{FinFun}$  are defined by mutual induction:

$$\begin{aligned} u, v, a, b &::= \perp \mid \mathbf{U} \mid \text{Fun}(g) \mid \text{Pi}(a, f) \\ g, f &\in \text{FinFun} = \text{List}(\text{FinEl} \times \text{FinEl}) \end{aligned}$$

Here  $\perp$  is the least element (no information),  $\mathbf{U}$  is the code for the universe,  $\text{Fun}(g)$  is a function element with graph  $g$  (a finite list of input–output pairs), and  $\text{Pi}(a, f)$  is a code for a  $\Pi$ -type with domain approximation  $a$  and codomain graph  $f$ .

**Binary supremum.** A partial binary supremum  $u \sqcup v$  is defined by case analysis: it combines compatible codes ( $\text{Pi}(a, f) \sqcup \text{Pi}(b, g) = \text{Pi}(a \sqcup b, f \mathbin{++} g)$ ,  $\text{Fun}(g) \sqcup \text{Fun}(h) = \text{Fun}(g \mathbin{++} h)$ ) and returns  $\perp$  for incompatible pairs (e.g.  $\mathbf{U} \sqcup \text{Fun}(g) = \perp$ ).

**Evaluation of finite functions.** Given a finite function  $f = [(u_1, v_1), \dots, (u_k, v_k)]$  and an input  $u$ , the sup-based evaluation collects all outputs whose keys are below  $u$ :

$$\text{ev}(f, u) = \bigsqcup \{v_i \mid u_i \leq u\}.$$

**Information ordering.** The ordering  $u \leq v$  on  $\text{FinEl}$  is defined mutually with an ordering on finite functions. The key clauses are:

$$\begin{aligned} \perp &\leq v = \top \\ \mathbf{U} &\leq \mathbf{U} = \top \\ \text{Fun}(g) &\leq \text{Fun}(h) \iff g \leq_{\text{fun}} h \\ \text{Pi}(a, f) &\leq \text{Pi}(b, g) \iff a \leq b \wedge f \leq_{\text{fun}} g \end{aligned}$$

where the ordering on finite functions uses the *finite condition*:

$$g \leq_{\text{fun}} h \iff \forall (u_i, v_i) \in g. v_i \leq \text{ev}(h, u_i).$$

All other combinations (e.g.  $\mathbf{U} \leq \text{Fun}(g)$ ) are empty.

**Coherence.** A finite element is *coherent* ( $\text{Coh}(u)$ ) when its internal data is consistent. Specifically,  $\text{Coh}(\text{Pi}(a, f))$  requires  $\text{Coh}(a)$  and that  $f$  is a *coherent finite function*: whenever two edges  $(u_1, v_1), (u_2, v_2) \in f$  have compatible keys ( $\text{Comp}(u_1, u_2)$ ), their values are also compatible ( $\text{Comp}(v_1, v_2)$ ). This condition ensures that finite functions represent well-defined continuous maps between domains.

**Finite membership.** The *finite typing* relation  $u : a$  (“ $u$  is an element of the type coded by  $a$ ”) is defined by mutual recursion on  $u$  and  $a$ :

$$\begin{aligned} \perp : a &\iff a : \mathbf{U} && \text{(every type has } \perp \text{)} \\ \mathbf{U} : \mathbf{U} &= \top \\ \text{Fun}(g) : \text{Pi}(a, f) &\iff \text{FinMemFun}(g, a, f) \wedge \text{Coh}(\text{Fun}(g)) \wedge \text{Pi}(a, f) : \mathbf{U} \\ \text{Pi}(a, f) : \mathbf{U} &\iff a : \mathbf{U} \wedge \text{FinMemAllU}(f, a) \wedge \text{CohFunTail}(f) \end{aligned}$$

where  $\text{FinMemFun}(g, a, f)$  checks that every edge  $(u_i, v_i) \in g$  satisfies  $u_i : a$  and  $v_i : \text{ev}(f, u_i)$ , and  $\text{FinMemAllU}(f, a)$  checks that every edge  $(a_i, b_i) \in f$  satisfies  $a_i : a$  and  $b_i : \mathbf{U}$ .

Key structural properties:

- $u : a$  implies  $\text{Coh}(u)$ ,  $\text{Coh}(a)$ , and  $a : \mathbf{U}$ .
- Membership is monotone: if  $u : a$  and  $a \leq a'$  with  $a' : \mathbf{U}$ , then  $u : a'$ .

## 7 Selections

A *selection* of a finite function  $f$  is a compatible subset of edges from  $f$ , yielding an aggregate key  $u$  and aggregate value  $v$ . Formally,  $\text{Selection}(f, u, v)$  is an inductive type with three constructors:

$$\begin{array}{c} \text{SEL-NIL} \\ \hline \text{Selection}(\text{nil}, \perp, \perp) \end{array} \qquad \begin{array}{c} \text{SEL-SKIP} \\ \text{Selection}(g, u, v) \\ \hline \text{Selection}((a_i, b_i)::g, u, v) \end{array}$$

$$\begin{array}{c} \text{SEL-TAKE} \\ \text{Comp}(a_i, u) \quad \text{Comp}(b_i, v) \quad \text{Selection}(g, u, v) \\ \hline \text{Selection}((a_i, b_i)::g, a_i \sqcup u, b_i \sqcup v) \end{array}$$

The compatibility conditions in SEL-TAKE ensure that the accumulated  $u$  and  $v$  remain coherent. A key property is that  $\text{Selection}(f, u, v)$  implies  $v \leq \text{ev}(f, u)$ .

Why selections rather than individual edges? The evaluation  $\text{ev}(f, u)$  is defined as a supremum: it aggregates the outputs of *all* edges in  $f$  whose keys lie below  $u$ . Working with single edges would not capture this aggregate character. A selection encodes a finite compatible subfamily of edges whose joins approximate  $\text{ev}(f, u)$  from below, and the compatibility check guarantees that these joins are well-defined (i.e. coherent). This is the mechanism that lets the relational semantics and the validity relation handle  $\Pi$ -types and  $\lambda$ -terms in a purely finite way, without ever computing an infinite supremum.

## 8 Relational semantics on finite approximants

**Definition 2** (Evaluation relation). *Given a raw term  $M$  in context of size  $n$ , a finite environment  $\rho$  (mapping each variable to a  $\text{FinEl}$ ), and a finite element  $u$ , the relation  $\llbracket M \rrbracket_\rho \ni u$  (“ $M$  evaluates to at least  $u$  in  $\rho$ ”) is defined by recursion on  $M$ :*

- **Variable:**  $\llbracket x_i \rrbracket_\rho \ni u$  iff  $\text{Coh}(u)$  and  $u \leq \rho(i)$ .
- **Universe:**  $\llbracket \mathbf{U} \rrbracket_\rho \ni u$  iff  $\text{Coh}(u)$  and  $u \leq \mathbf{U}$ .

- **Application:**  $\llbracket M \ N \rrbracket_\rho \ni \perp$  always holds. For non- $\perp$  output  $u$ : there exists  $v$  such that  $\llbracket N \rrbracket_\rho \ni v$  and  $\llbracket M \rrbracket_\rho \ni \text{Fun}([(v, u)])$ . (One-edge form: the function  $M$  maps input  $v$  to output  $u$ .)
- **Lambda:**  $\llbracket \lambda(x:A)M \rrbracket_\rho \ni \perp$  always holds.  $\llbracket \lambda(x:A)M \rrbracket_\rho \ni \text{Fun}(g)$  requires a domain code  $a$  with  $\text{Coh}(\text{Fun}(g))$ ,  $a : \mathbf{U}$ ,  $\llbracket A \rrbracket_\rho \ni a$ , and for every selection of  $g$  with key  $u$  and value  $v$ , there exists  $x \leq u$  with  $x : a$  and  $\llbracket M \rrbracket_{\rho[x]} \ni v$ .
- **Pi:**  $\llbracket \Pi(x:A)B \rrbracket_\rho \ni \perp$  always holds.  $\llbracket \Pi(x:A)B \rrbracket_\rho \ni \text{Pi}(a, f)$  requires  $\text{Coh}(\text{Pi}(a, f))$ ,  $\llbracket A \rrbracket_\rho \ni a$ , and a witness  $a'$  with  $\llbracket A \rrbracket_\rho \ni a'$  such that for every selection of  $f$  with key  $u$  and value  $v$ , there exists  $x \leq u$  with  $x : a'$  and  $\llbracket B \rrbracket_{\rho[x]} \ni v$ .

The relation is purely finite and has no reference to ideal semantic functions. Key properties:

- *Coherence extraction:*  $\llbracket M \rrbracket_\rho \ni u$  implies  $\text{Coh}(u)$ .
- *Compatibility:* if  $\llbracket M \rrbracket_\rho \ni u$  and  $\llbracket M \rrbracket_\rho \ni v$  with  $\text{Coh}(\rho)$ , then  $\text{Comp}(u, v)$ .
- *Monotonicity in  $\rho$ :* if  $\rho \leq \rho'$  pointwise and  $\llbracket M \rrbracket_\rho \ni u$ , then  $\llbracket M \rrbracket_{\rho'} \ni u$ .
- *Downward closure:*  $\llbracket M \rrbracket_\rho \ni u$  and  $u' \leq u$  with  $\text{Coh}(u')$  implies  $\llbracket M \rrbracket_\rho \ni u'$ .
- *Sup closure:*  $\llbracket M \rrbracket_\rho \ni u$  and  $\llbracket M \rrbracket_\rho \ni v$  implies  $\llbracket M \rrbracket_\rho \ni u \sqcup v$  (when the environment is coherent).

## 9 Soundness of the relational semantics

**Definition 3** (Fitting environment). *An environment  $\rho$  fits a context  $\Gamma$  (written  $\text{Fits}(\Gamma, \rho)$ ) when, for each variable  $x_i : A_i$  in  $\Gamma$ , there exists a code  $a'$  such that  $\rho(i) : a'$  and  $\llbracket A_i \rrbracket_{\rho \upharpoonright i} \ni a'$ .*

**Definition 4** (Typed evaluation).  *$\text{Typed}(M, A, \rho, u)$  asserts that  $u$  can be enlarged to some  $u' \geq u$  which is a member of some code  $a'$  that  $A$  evaluates to: there exist  $u'$  and  $a'$  such that  $u \leq u'$ ,  $\llbracket M \rrbracket_\rho \ni u'$ ,  $u' : a'$ , and  $\llbracket A \rrbracket_\rho \ni a'$ .*

**Theorem 5** (Typing soundness — Theorem 1 of [1]). *If  $\Gamma \vdash M : A$  and  $\text{Fits}(\Gamma, \rho)$ , then for every  $u$  with  $\llbracket M \rrbracket_\rho \ni u$ , we have  $\text{Typed}(M, A, \rho, u)$ .*

**Theorem 6** (Conversion soundness). *If  $\Gamma \vdash M = N : A$  and  $\text{Fits}(\Gamma, \rho)$ , then:*

1. *Typing:*  $\text{Typed}(M, A, \rho, u)$  for all  $\llbracket M \rrbracket_\rho \ni u$ , and likewise for  $N$ .
2. *Bidirectional transfer:* for all  $u$ ,  $\llbracket M \rrbracket_\rho \ni u \iff \llbracket N \rrbracket_\rho \ni u$ .

The proofs proceed by mutual induction on the typing and conversion derivations. The  $\beta$  and function extensionality cases use the evaluation–substitution lemma: if  $\llbracket M \rrbracket_{\rho[v]} \ni u$  and  $\llbracket N \rrbracket_\rho \ni v$ , then  $\llbracket M[N/x] \rrbracket_\rho \ni u$  (and conversely). Formalised in `TypingSemantics.agda` and `LemmaForTS.agda`, with 0 postulates.

## 10 Logical relation (validity)

The validity relation is defined by mutual structural recursion on the type code  $a \in \text{FinEl}$ . It connects the syntactic judgments to the finite semantics.

**Definition 7** (Validity). *For a context  $\Gamma$ , terms  $M, A$ , a realizer  $u$ , and a type code  $a$ , we define mutually:*

**Term validity**  $\text{Val}(\Gamma, M, A, u, a)$  by case split on  $a$ :

- At  $a = \perp$ : always  $\top$  (trivially valid).
- At  $a = \text{U}$ : reduces to type validity  $\text{ValTy}(\Gamma, M, u)$ .
- At  $a = \text{Pi}(b, f)$  with  $u = \text{Fun}(g)$ : a  $\Sigma$ -type containing
  1. type validity of  $A$  at  $\text{Pi}(b, f)$  (head-reduces to some  $\Pi A_0 B_0$ ),
  2.  $\text{Coh}(\text{Fun}(g))$  and  $\text{FinMemFun}(g, b, f)$ ,
  3. for every selection of  $g$  with key  $u'$  and value  $v$ , and argument  $N$  with  $\text{Val}(\Gamma, N, A_0, u', b)$ :  

$$\text{Val}(\Gamma, M \ N, B_0[N], v, \text{ev}(f, u'))$$
.
- At  $a = \text{Pi}(b, f)$  with  $u \neq \text{Fun}(g)$ : always  $\top$ .

**Type validity**  $\text{ValTy}(\Gamma, M, u)$  at  $u = \text{Pi}(b, f)$ : a  $\Sigma$ -type containing terms  $A, B$  such that  $M \rightsquigarrow^* \Pi(x:A)B$  in  $\Gamma$ ,  $\text{CohFunTail}(f)$ ,  $\text{FinMemAllU}(f, b)$ , type validity  $\text{ValTy}(\Gamma, A, b)$ , and for every selection of  $f$  with key  $u'$  and value  $v$ , and argument  $N$  with  $\text{Val}(\Gamma, N, A, u', b)$ :  $\text{ValTy}(\Gamma, B[N], v)$ .

**Equality validity**  $\text{EqVal}(\Gamma, M, N, A, u, a)$  bundles validity of both  $M$  and  $N$ , plus type-equality data at  $\Pi$ -codes.

The key feature of this logical relation is that it recurses on the type code  $a$  rather than on syntactic types. Since  $\text{ev}(f, u')$  is structurally smaller than  $\text{Pi}(b, f)$  (its rank is bounded by  $\text{rk}_{\text{fun}}(f) < \text{rk}(\text{Pi}(b, f))$ ), the recursion is well-founded despite the type-in-type universe.

**Monotonicity properties.** Three mutual monotonicity families are proved:

- *Downward in  $a$* : if  $a_0 \leq a_1$ ,  $u : a_0$ ,  $\text{Coh}(a_0)$ , and  $a_1 : \text{U}$ , then  $\text{Val}(\Gamma, M, A, u, a_1) \Rightarrow \text{Val}(\Gamma, M, A, u, a_0)$ .
- *Upward in  $a$* : if  $a_0 \leq a_1$ ,  $u : a_0$ ,  $u : a_1$ ,  $\text{Coh}(a_0)$ ,  $\text{Coh}(a_1)$ , and  $\text{ValTy}(\Gamma, A, a_1)$ , then  $\text{Val}(\Gamma, M, A, u, a_0) \Rightarrow \text{Val}(\Gamma, M, A, u, a_1)$ .
- *Restriction in  $u$* : if  $u' \leq u$ ,  $u' : a$ , and  $u : a$ , then  $\text{Val}(\Gamma, M, A, u, a) \Rightarrow \text{Val}(\Gamma, M, A, u', a)$ .

Formalised in `Validity.agda` with 0 postulates.

## 11 Adequacy

The adequacy theorem is the bridge between the syntactic typing judgments and the semantic validity relation. It is proved using the two-substitution technique of [1], Theorem 2.

**Definition 8** (Bundled validity).  $\text{Val2}(\Gamma, M, A, u, a)$  refines  $\text{Val}$  by requiring  $\Gamma \vdash M : A$  at the leaves (at  $\perp$ -realizer or  $\perp$ -code positions). Similarly,  $\text{EqVal2}$  requires  $\Gamma \vdash M = N : A$ . These bundled versions are defined in `Validity2.agda`.

**Theorem 9** (Adequacy — two-substitution form). *The following are proved by mutual induction on typing/conversion derivations, instantiated at two substitutions  $\sigma_1, \sigma_2$  simultaneously:*

1. If  $\Gamma \vdash M : A$ , then for all codes  $u, a$  with  $\llbracket M \rrbracket_\rho \ni u$  and  $\llbracket A \rrbracket_\rho \ni a$ :

$$\text{Val2}(\Delta, M[\sigma], A[\sigma], u, a).$$

2. If  $\Gamma \vdash M = N : A$ , then under the same conditions:

$$\text{EqVal2}(\Delta, M[\sigma], N[\sigma], A[\sigma], u, a).$$

The proof uses the substitution lemma (well-typed substitutions preserve typing and conversion), context conversion, the soundness theorems 5 and 6, and the monotonicity properties of the validity relation. Key cases:

- BETA: uses the evaluation–substitution correspondence and  $\beta$ -expansion/contraction lemmas for Val2.
- FUNEXT: uses weakening for EvalRel and the selection-based Pi clauses.
- CONV: uses the bidirectional transfer from conversion soundness and the downward/upward monotonicity of Val2.
- PI: constructs a  $\text{Pi}(b, f)$ -realizer by building selections from the domain and codomain adequacy hypotheses.

Formalised in `Adequacy2.agda` with 0 postulates.

## 12 Main result: Pi injectivity

**Theorem 10** (Corollary 6; Coquand–Huber 2018, p. 661). *If  $\Gamma \vdash A_0 = \Pi(x:B_1)F_1 : \mathbb{U}$ , then there exist  $B_0, F_0$  such that*

1.  $A_0 \rightsquigarrow^* \Pi(x:B_0)F_0$ ,
2.  $\Gamma \vdash B_0 = B_1 : \mathbb{U}$ ,
3.  $\Gamma, x:B_0 \vdash F_0 = F_1 : \mathbb{U}$ .

**Corollary 11** (Pi–Pi injectivity). *If  $\Gamma \vdash \Pi(x:A_0)B_0 = \Pi(x:A_1)B_1 : \mathbb{U}$ , then*

$$\Gamma \vdash A_0 = A_1 : \mathbb{U} \quad \text{and} \quad \Gamma, x:A_0 \vdash B_0 = B_1 : \mathbb{U}.$$

*Proof outline (Theorem).* The proof proceeds by domain-theoretic adequacy.

1. From  $\Gamma \vdash A_0 = \Pi B_1 F_1 : \mathbb{U}$ , extract  $\Gamma \vdash A_0 : \mathbb{U}$  by presupposition (SUBSTITUTIONLEMMA).
2. Apply the soundness theorem (CONVSOUND') at the *bottom environment*  $\rho_\perp$  (mapping every variable to  $\perp$ ) to obtain a bidirectional evaluation transfer: for every finite approximation  $u$ ,

$$\text{EvalRel}(\Pi B_1 F_1, \rho_\perp, u) \implies \text{EvalRel}(A_0, \rho_\perp, u).$$

3. Instantiate with  $u = \text{PiCode}(\perp, \text{nil})$ , which is coherent because  $\text{CoherentFunTail}(\text{nil}) = \top$ . The evaluation  $\text{EvalRel}(\Pi B_1 F_1, \rho_\perp, \text{PiCode}(\perp, \text{nil}))$  is constructible trivially (domain  $\perp$ , empty function graph, vacuous selection body). Transfer gives  $\text{EvalRel}(A_0, \rho_\perp, \text{PiCode}(\perp, \text{nil}))$ .
4. Apply the bundled equality adequacy theorem (`ADEQUACYEQSUB2`) with the identity substitution at  $\rho_\perp$  to obtain

$$\text{EqVal2 } \Gamma \ A_0 \ (\Pi B_1 F_1) \ \cup \ (\text{PiCode}(\perp, \text{nil})) \ \cup \text{Code}.$$

5. This unfolds to  $\text{EqValTyPi2}$ , a  $\Sigma$ -type containing:

- $\text{Red } \Gamma \ A_0 \ (\Pi B_0 F_0) \ \cup$  (giving part 1),
- $\Gamma \vdash B_0 = B'_1 : \mathcal{U}$  and  $\Gamma, x:B_0 \vdash F_0 = F'_1 : \mathcal{U}$ ,
- $\text{Red } \Gamma \ (\Pi B_1 F_1) \ (\Pi B'_1 F'_1) \ \cup$ .

Since  $\Pi$  is a head normal form,  $\text{Red-unique-Pi}$  forces  $B'_1 = B_1$  and  $F'_1 = F_1$ , giving parts 2 and 3. □

*Proof of Corollary.* Apply the theorem to  $\Gamma \vdash \Pi A_0 B_0 = \Pi A_1 B_1 : \mathcal{U}$ . This yields  $B'_0, F'_0$  with  $\Pi A_0 B_0 \rightsquigarrow^* \Pi B'_0 F'_0$ . Since  $\Pi$  is a head normal form,  $\text{Red-unique-Pi}$  gives  $B'_0 = A_0$  and  $F'_0 = B_0$ , so the domain and codomain conversions become  $\Gamma \vdash A_0 = A_1 : \mathcal{U}$  and  $\Gamma, x:A_0 \vdash B_0 = B_1 : \mathcal{U}$ . □

**Key intermediate lemmas.** The proof above relies on several results that deserve explicit mention:

- *Presupposition* (`SubstitutionLemma.agda`): from a conversion judgment  $\Gamma \vdash M = N : A$  one can extract  $\Gamma \vdash M : A$ ,  $\Gamma \vdash N : A$ , and  $\Gamma \vdash A : \mathcal{U}$ . This is proved by mutual induction on typing and conversion, using the substitution and renaming lemmas.
- *Conversion soundness* (`convSound'` in `TypingSemantics.agda`): if  $\Gamma \vdash M = N : A$  and  $\text{EvalRel}(N, \rho, u)$ , then  $\text{EvalRel}(M, \rho, u)$  (and vice versa).
- *Bundled adequacy* (`adequacyEqSub2` in `Adequacy2.agda`): the bridge from  $\text{EvalRel}$  to the validity relation  $\text{EqVal2}$ , which packages both a semantic value and the corresponding syntactic typing/conversion derivation.
- *Reduction uniqueness for  $\Pi$*  (**Red-unique-Pi**): since  $\Pi$  is a head normal form,  $\Pi A B \rightsquigarrow^* \Pi A' B'$  implies  $A = A'$  and  $B = B'$ . This is the step that turns the “up to reduction” output of adequacy into exact equalities on the domain and codomain.

The entire proof is formalised in Agda with `--without-K` `--exact-split`, no `--type-in-type`, and no postulates (in the files on which `PiInjectivity.agda` depends).

## References

- [1] T. Coquand and S. Huber, “An adequacy theorem for dependent type theory,” *Archive for Mathematical Logic*, vol. 57, pp. 649–676, 2018.